



M. Di Ianni

Lezione 11 – modelli di calcolo

Lezioni del 26/03/2026



PascalMinimo e macchine di Turing

- Obiettivo di questa lezione è dimostrare il teorema inverso del teorema 3.5, ossia il seguente
- **Teorema 3.6:** Per ogni macchina di Turing deterministica T di tipo riconoscitore ad un nastro esiste un programma $\mathcal{A}_T$ scritto in accordo alle regole del linguaggio PascalMinimo tale che: per ogni parola x , $T(x)$ termina nello stato di accettazione q_A se e soltanto se l'esecuzione di $\mathcal{A}_T$ con input x restituisce q_A in output

PascalMinimo e macchine di Turing

- Il nostro obiettivo sarà raggiunto progettando un algoritmo in PascalMinimo che simula la macchina di Turing Universale
 - lo trovate nella dispensa 3, nelle ultime 3 righe a pag. 9, a pag. 11, e nella Tabella 3.3 a pag. 12
 - è poco più di un esercizio
- Utilizziamo questo esercizio per dimostrare il Teorema 3.6
 - che nella dispensa è dimostrato in maniera leggermente diversa (anche se la tecnica è la stessa)
- Infatti, se progettiamo un programma $\mathcal{U}$ in PascalMinimo che simula la macchina di Turing Universale
 - ossia, tale che l'esecuzione di $\mathcal{U}$ con input la descrizione di una macchina di Turing T e un input x per T simula la computazione $U(T,x)$
- otteniamo *un programma capace di simulare qualunque macchina di Turing*

PascalMinimo e macchine di Turing

- **Teorema 3.6:** Per ogni macchina di Turing deterministica T di tipo riconoscitore ad un nastro esiste un programma $\mathcal{A}_T$ scritto in accordo alle regole del linguaggio PascalMinimo tale che: per ogni parola x , $T(x)$ termina nello stato di accettazione q_A se e soltanto se l'esecuzione di $\mathcal{A}_T$ con input x restituisce q_A in output
- Dimostriamo questo teorema progettando un programma $\mathcal{U}$ che si comporta come la macchina Universale:
 - utilizzando opportune strutture dati per rappresentare le quintuple di una generica macchina di Turing, e il suo stato iniziale, e i suoi stati finali
 - e altre opportune strutture dati per rappresentare un input di quella generica macchina,
 - fornendo in input ad $\mathcal{U}$ le descrizioni di una data macchina T e di un suo dato input x (in accordo alle strutture dati utilizzate)
 - l'esecuzione di $\mathcal{U}$ sul suo input restituisce un output che corrisponde allo stato in cui terminerebbe la computazione $T(x)$
 - o che non termina qualora $T(x)$ non terminasse

PascalMinimo e macchine di Turing

- Progettiamo un programma $\mathcal{U}$ che si comporta come la macchina Universale:
 - per memorizzare le quintuple della macchina T che si vuole simulare, utilizziamo i 5 array $Q1, S1, S2, Q2, M$:
 - e usiamo i valori $-1, 0, 1$ per rappresentare i movimenti della testina 'sinistra', 'ferma', 'destra'
 - se la i -esima quintupla di T è $\langle q, a, b, q', \text{sinistra} \rangle$, allora avremo
 $Q1[i] = q, S1[i] = a, S2[i] = b, Q2[i] = q', M[i] = -1$
 - e analogamente per i movimenti della testina 'ferma' e 'destra'
 - $Q1[i]$ memorizza lo stato in cui si deve trovare la macchina per eseguire la quintupla i , $Q2[i]$ memorizza lo stato in cui deve entrare la macchina dopo aver eseguito la quintupla i , e analogamente per $S1[i]$, $S2[i]$ e $M[i]$
 - rappresentiamo il nastro di T mediante l'array N , che, per semplicità, ammettiamo possa avere anche indici negativi
 - ad esempio, $N[-4]$: tanto il PascalMinimo ce lo stiamo inventando...
- A questo punto, vediamo il programma, nel prossimo lucido
 - che trovate (spiegato nel dettaglio) anche sulla dispensa!**

Un programma che simula U

In input viene fornita la descrizione di T (negli array Q_1 , S_1 , S_2 , Q_2 e M e nelle variabili q_0 , q_A e q_R) e del suo input (nell'array N)

Input: stringa $x_1x_2\dots x_n$ memorizzata nell'array N , con $N[i] = x_i$ per $i = 1, \dots, n$, array Q , Σ , Q_1 , S_1 , S_2 , Q_2 , M descritti nel testo, il numero k di quintuple, q_0 , q_A , q_R .

```
1   $q \leftarrow q_0$ ;  
2   $t \leftarrow 1$ ;  
3   $\text{primaCella} \leftarrow 1$ ;  
4   $\text{ultimaCella} \leftarrow n$ ;  
5  while ( $q \neq q_A \wedge q \neq q_R$ ) do begin  
6       $j \leftarrow 1$ ;  
7       $\text{trovata} \leftarrow \text{falso}$ ;  
8      while ( $j \leq k \wedge \text{trovata} = \text{falso}$ ) do  
9          if ( $q = Q_1[j] \wedge N[t] = S_1[j]$ ) then  $\text{trovata} \leftarrow \text{vero}$ ;  
10         else  $j \leftarrow j + 1$ ;  
11     if ( $\text{trovata} = \text{vero}$ ) then begin  
12          $N[t] \leftarrow S_2[j]$ ;  
13          $q \leftarrow Q_2[j]$ ;  
14          $t \leftarrow t + M[j]$ ;  
15         if ( $t < \text{primaCella}$ ) then begin  
16              $\text{primaCella} \leftarrow t$ ;  
17              $N[t] \leftarrow \square$ ;  
18         end  
19         if ( $t > \text{ultimaCella}$ ) then begin  
20              $\text{ultimaCella} \leftarrow t$ ;  
21              $N[t] \leftarrow \square$ ;  
22         end  
23     end  
24     else  $q \leftarrow q_R$ ;  
25 end  
26 Output:  $q$ 
```

q è la variabile in cui memorizziamo lo stato interno di T
e t è la variabile in cui memorizziamo la posizione della testina di T ad ogni passo della computazione

Il programma consiste in un loop while che termina quando viene raggiunto uno stato finale

NB: k è il numero di quintuple della macchina T che si vuole simulare

Un programma che simula U

```
5  while ( $q \neq q_A \wedge q \neq q_R$ ) do begin
6       $j \leftarrow 1$ ;
7      trovata  $\leftarrow$  falso;
8      while ( $j \leq k \wedge$  trovata = falso) do
9          if ( $q = Q_1[j] \wedge N[t] = S_1[j]$ ) then trovata  $\leftarrow$  vero;
10         else  $j \leftarrow j + 1$ ;
11     if (trovata = vero) then begin
12          $N[t] \leftarrow S_2[j]$ ;
13          $q \leftarrow Q_2[j]$ ;
14          $t \leftarrow t + M[j]$ ;
15         if ( $t < primaCella$ ) then begin
16             primaCella  $\leftarrow$  t;
17              $N[t] \leftarrow \square$ ;
18         end
19         if ( $t > ultimaCella$ ) then begin
20             ultimaCella  $\leftarrow$  t;
21              $N[t] \leftarrow \square$ ;
22         end
23     end
24     else  $q \leftarrow q_R$ ;
25 end
```

vengono esaminate ordinatamente,
dalla prima ($j=1$) all'ultima ($j=k$),
le quintuple di T
fino a quando:

ne viene trovata una che può essere
eseguita (quando $Q_1[j]=q$ e $S_1[j] = N[t]$)
e in questo caso si pone
trovata = true

oppure non ne viene trovata alcuna
che può essere eseguita (quando
 $j = k+1$)

Un programma che simula U

```
3  primaCella ← 1;  
4  ultimaCella ← n;
```

...

```
11  if (trovata = vero) then begin
```

```
12      N[t] ← S2[j];
```

```
13      q ← Q2[j];
```

```
14      t ← t + M[j];
```

```
15      if (t < primaCella) then begin
```

```
16          primaCella ← t;
```

```
17          N[t] ← □;
```

```
18      end
```

```
19      if (t > ultimaCella) then begin
```

```
20          ultimaCella ← t;
```

```
21          N[t] ← □;
```

```
22      end
```

```
23  end
```

in questo modo memorizziamo gli indici dell'array *N* che individuano, rispettivamente, la cella più a sinistra e la cella più a destra della porzione di nastro non blank di *T*

Se la quintupla da eseguire è stata trovata, la si esegue: si aggiorna l'elemento *N*[*t*], si aggiorna lo stato interno *q*, si muove la testina (*t* + *M*[*j*])

se, eseguendo la quintupla, la testina di *T* viene spostata su una cella a sinistra della porzione di nastro sinora utilizzata (*t* < *primaCella*) oppure su una cella a destra della porzione di nastro sinora utilizzata (*t* > *ultimaCella*) allora *primaCella* o *ultimaCella* devono essere aggiornate

Un programma che simula U

```
5   while ( $q \neq q_A \wedge q \neq q_R$ ) do begin
6        $j \leftarrow 1$ ;
7       trovata  $\leftarrow$  falso;
8       while ( $j \leq k \wedge$  trovata = falso) do
9           if ( $q = Q_1[j] \wedge N[t] = S_1[j]$ ) then trovata  $\leftarrow$  vero;
10          else  $j \leftarrow j + 1$ ;
11      if (trovata = vero) then begin
12          ...
23      end
24      else  $q \leftarrow q_R$ ;
25  end
26  Output:  $q$ 
```

se non viene trovata alcuna quintupla da eseguire, si porta la macchina nello stato di rigetto

Ricordate quello che avevamo detto a proposito di un insieme di quintuple **non** completo?



Macchina non deterministica in PascalMinimo

- Guardiamo insieme ora l'algoritmo in PascalMinimo che simula una macchina di Turing non deterministica (che trovate al paragrafo 3.4)
 - meno facile rispetto alla simulazione della macchina Universale...
- L'algoritmo implementa in PascalMinimo la coda di rondine con ripetizioni che abbiamo descritto informalmente nel corso della Lezione 4, e che dimostra il Teorema 2.1 (Dispensa 2):
 - inizializza un contatore i a 1
 - simula **tutte** le computazioni deterministiche di i istruzioni
 - se una di esse accetta, allora accetta
 - altrimenti; se tutte rigettano, allora rigetta
 - se al passo precedente non hai terminato (ossia, nessuna computazione di i passi ha accettato e almeno una di esse non ha rigettato), allora incrementa il valore di i e ripeti il passo precedente

Macchina non deterministica in PascalMinimo

- e tutto ciò si traduce in PascalMinimo nel programma seguente

Input: stringa $x_1x_2 \dots x_n$ memorizzata nell'array N , con $N[i] = x_i$ per $i = 1, \dots, n$.
Il programma utilizza anche le seguenti costanti: q_0, q_A, q_R e inoltre
gli array $Q, \Sigma, Q_1, S_1, S_2, Q_2, M$ descritti nel testo, il numero k di quintuple,

```
1   $q \leftarrow q_0$ ;  
2   $i \leftarrow 1$ ;  
3   $\text{primaCella} \leftarrow 1$ ;  
4   $\text{ultimaCella} \leftarrow n$ ;  
5  while ( $q \neq q_A \wedge q \neq q_R$ ) do begin  
6       $q \leftarrow \text{simulaRicorsivo}(q_0, 1, N, i)$ ;  
7       $i \leftarrow i + 1$ ;  
8  end  
9  Output:  $q$ 
```

*primaCella e ultimaCella
sono variabili globali*

- ove la simulazione di tutte le computazioni deterministiche è eseguita dall'invocazione della funzione ricorsiva $\text{simulaRicorsivo}(q_0, 1, N, i)$
 - i cui parametri sono: lo stato interno (q_0), la posizione della testina (1) e il contenuto del nastro (N) della macchina non deterministica quando ha inizio la simulazione, e la lunghezza (i) delle computazioni da simulare

Macchina non deterministica in PascalMinimo

- Ecco lo schema della funzione ricorsiva:

```
Q simulaRicorsivo(Q q, int t,  $\Sigma$ [ ] N, int i)
```

```
begin
```

```
  if (i = 0) then  $\rho \leftarrow q$ ;
```

```
  else begin
```

per ogni quintupla che puoi eseguire a partire dallo stato globale in cui ti trovi
esegui e fai partire tutte le simulazioni delle computazioni lunghe $i-1$

(invocazione ricorsiva di simulaRicorsivo):

se **una di esse** restituisce q_A (ossia, accetta) allora termina

le simulazioni con $\rho = q_A$

altrimenti, se almeno una simulazione **non** restituisce q_R (ossia, non rigetta)

allora poni rigetto $\leftarrow$ falso per ricordartelo

altrimenti, se **tutte** le simulazioni restituiscono q_R (ossia, rigettano)

allora avrai $\rho = q_R$

```
  end
```

```
  if ( $\rho \neq q_A$  e rigetto = falso) then  $\rho \leftarrow q_0$ ;
```

```
  return  $\rho$ 
```

```
end
```

```

1  Q funzione simulaRicorsivo(Q q, int t,  $\Sigma[\ ]$  N, int i)
2  begin
3      if (i = 0) then  $\rho \leftarrow q$ ;
4          // la precedente istruzione gestisce il caso in cui la ricorsione termina
5      else begin
6          j  $\leftarrow$  1;
7          rigetto  $\leftarrow$  vero;
8          while ( $j \leq k \wedge q \neq q_A$ ) do begin
9              while ( $j \leq k \wedge [q \neq Q_1[j] \vee N[t] \neq S_1[j]]$ ) do j  $\leftarrow$  j + 1;
10             if (j = k + 1) then  $\rho \leftarrow q_R$ ;
11                 // nessuna (ulteriore) quintupla da eseguire è stata trovata
12             else begin
13                 // ora i primi due elementi di  $p_j$  sono q e N[t]
14                 N[t]  $\leftarrow$  S2[j];
15                 if (t + M[j] < primaCella) then begin
16                     primaCella  $\leftarrow$  t + M[j];
17                     N[t + M[j]]  $\leftarrow$  □;
18                 end
19                 if (t + M[j] > ultimaCella) then begin
20                     ultimaCella  $\leftarrow$  t + M[j];
21                     N[t + M[j]]  $\leftarrow$  □;
22                 end
23                  $\rho \leftarrow$  simulaRicorsivo(Q2[j], t + M[j], N, i - 1);
24                 if ( $\rho = q_A$ ) then  $q \leftarrow q_A$ ;
25                 else if ( $\rho \neq q_R$ ) then rigetto  $\leftarrow$  falso;
26                 N[t]  $\leftarrow$  S1[j];
27                     // lo stato di N viene ripristinato alla situazione precedente l'esecuzione
28                     // della quintupla  $p_j$ 
29                 j  $\leftarrow$  j + 1;
30                 // si predispone ad iniziare la ricerca di una nuova quintupla da eseguire:
31                 // tale ricerca avrà luogo solo se nessuna computazione deterministica
32                 // precedente (iniziata da una quintupla  $p_h$  con  $h < j$ ) ha accettato
33             end;
34         end;
35     if ( $\rho \neq q_A \wedge \text{rigetto} = \text{falso}$ ) then  $\rho \leftarrow q_0$ ;
36         // la precedente istruzione gestisce il caso in cui il ciclo while alle linee 6-25
37         // termina senza aver trovato lo stato  $q_A$  e senza poter decidere circa il rigetto;
38         // si osservi che, se  $\rho \neq q_A$  e rigetto = vero, allora tutte le quintuple eseguite
39         // hanno portato a rigettare e, quindi,  $\rho = q_R$ 
40     end;
41 return:  $\rho$ ;
42 end

```

primaCella e *ultimaCella*
sono variabili globali